VECTOR



SOME/IP with CANoe Communication Setup

Part 2. Data Source – vCDL

V1.0 | 2023-07-18

Agenda

- **Overview**
- Data Source (vCDL)
- Technical References

This document is for …

- Help you write SOME/IP service description with vCDL

- Help to understand for the Initial Setup
  - SOME/IP concept in CANoe
  - Import Data Sources
  - Import Application Models

  covered in Part1.

  - Write Data Source as a vCDL
  - Simulation with CANoe GUI features
  - Analysis of SOME/IP data
  - Simulation with CAPL programming

  will be covered in other Parts…

- CANoe sample configuration is prepared for your understanding
  - https://portal.vector.com/shared/068f1303-77de-4eab-8bd0-ca37938c6885

- Quick start guide for work of basic SOME/IP simulation, analysis
  - You can extend your knowledge and know-how through CANoe Help Manual!
  - It covers More detailed technical information.

Agenda

Overview

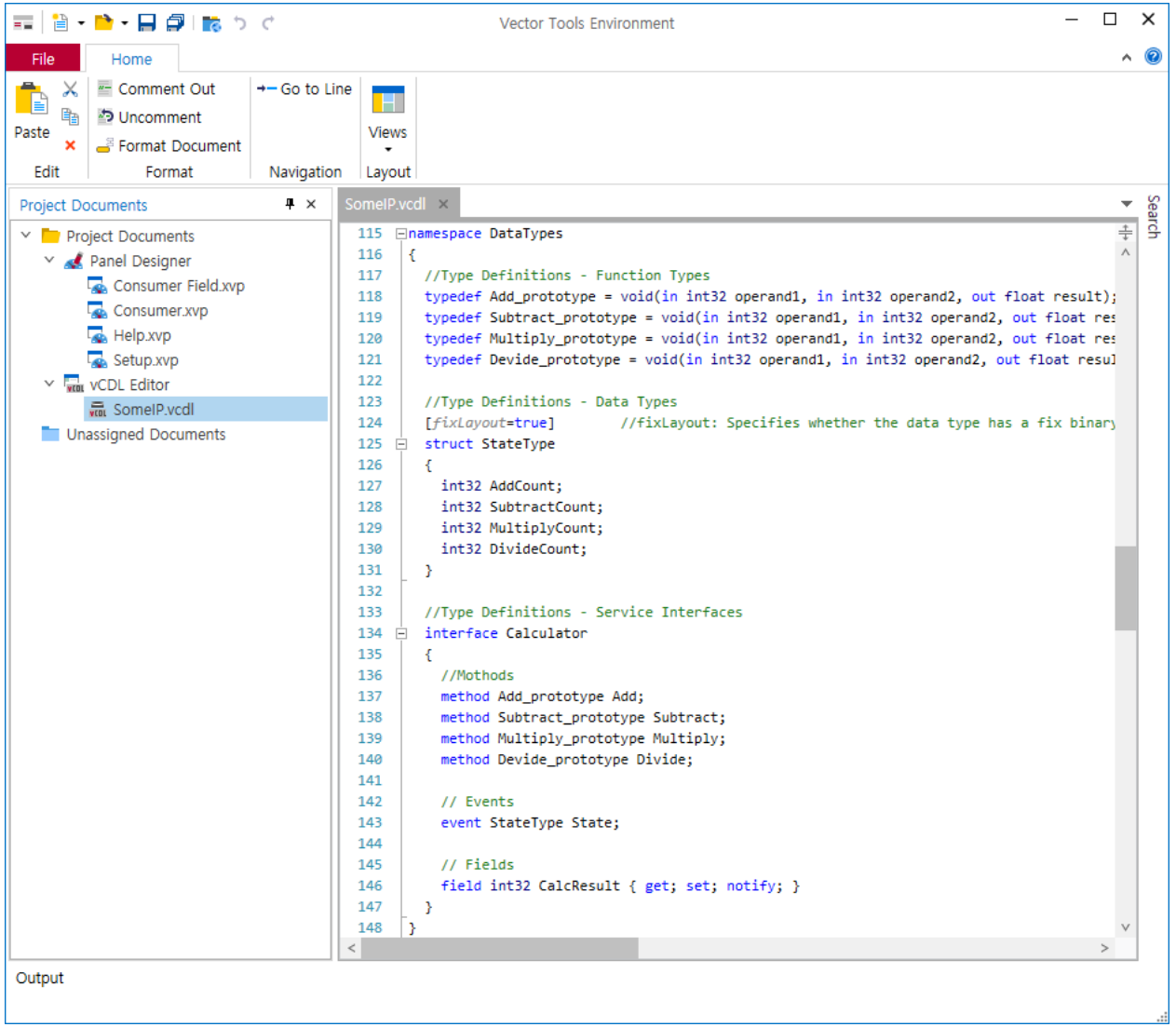
- **Data Source (vCDL)**

Technical References

vCDL: Vector Communication Description Language

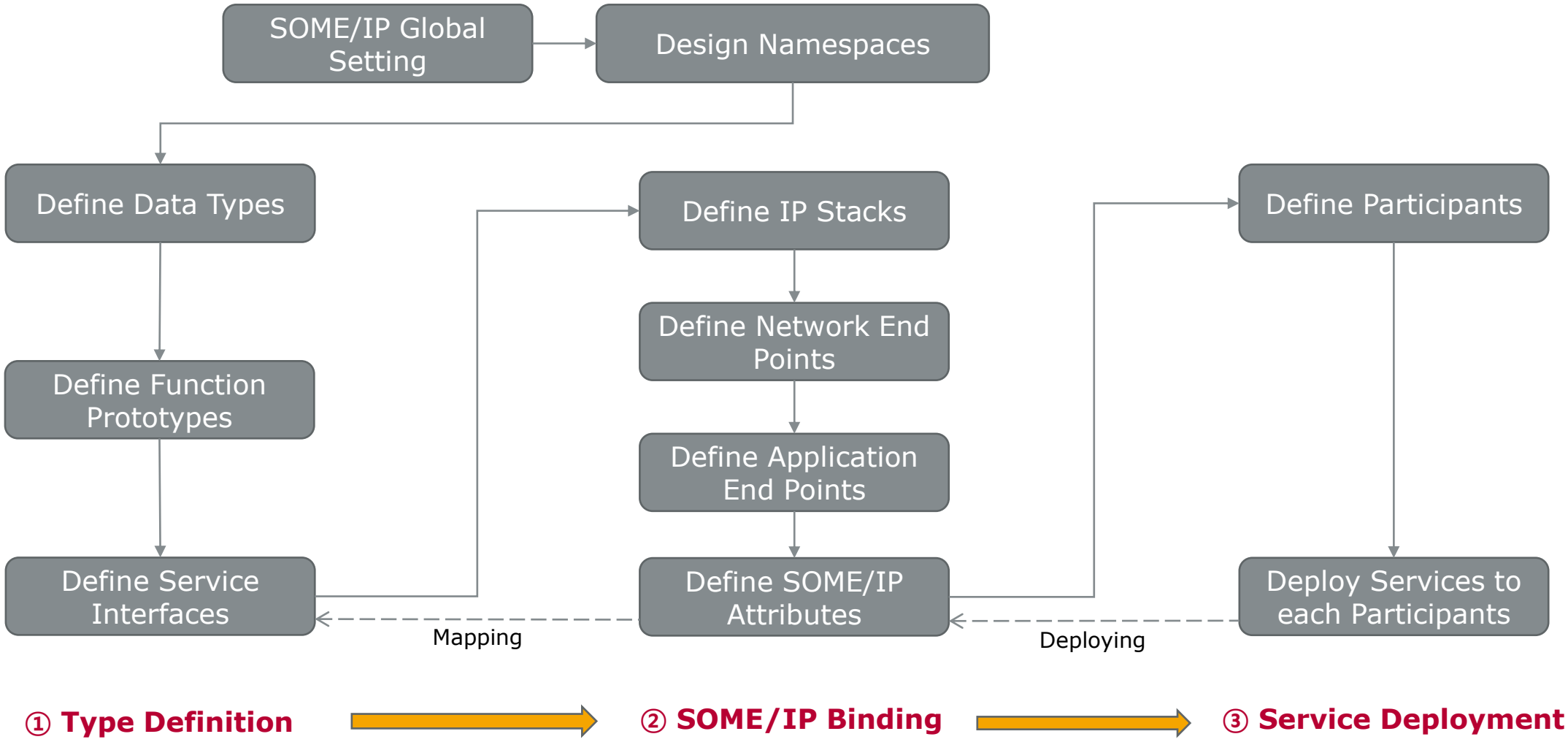
- **vCDL:**
  - Used to describe Communication and Distributed Objects
  - Is independent from OEM's description file type and version (ARXML, FIBEX, …)
  - Allows full or complementary description of missing system information in OEM's description files

- **vCDL Editor:**
  - vCDL Editor is integrated in the Vector Tools Environment by now
  - The integration allows an easy and fast access to all assigned vCDL files in a configuration
  - The Editor is used to create or to edit vCDL files



vCDL: Vector Communication Description Language

- **Recommended vCDL Workflow for SOME/IP Configuration**



vCDL: Vector Communication Description Language

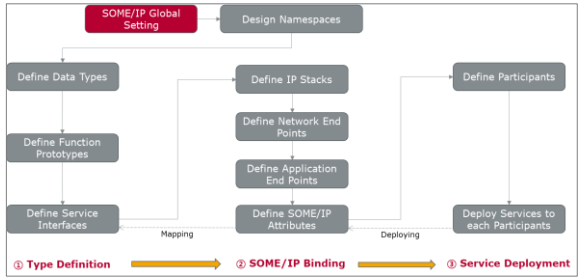
- SOME/IP Global Setting
  - Define SOME/IP Communication Attributes
  - Service Discovery
    - The SOME/IP service discovery needs to be configured with some global parameters, controlling how often and when service discovery messages are sent.
  - Serialization Configuration
    - For SOME/IP bound data transmission it needs to be specified how that data is serialized on the network.

```
[
// Partial Rest Bus Simulation(PRBS)
  simulationChannel=1
, prbsLayer="Ethernet"

// SD Server Settings
, offerTTL=10s
, serverResponseDelayMin=0s
, serverResponseDelayMax=0s
, initialOfferDelayMin=0s
, initialOfferDelayMax=0s
, offerRepetitionsBaseDelay=0.15s
, offerRepetitionsMax=2
, offerCycleTime=3s

// SD Client Settings
, findTTL=10s
, clientResponseDelayMin=0s
, clientResponseDelayMax=0s
, findRepetitionsMax=5
, findRepetitionsBaseDelay=0.02s
, initialFindDelayMax=0s
, initialFindDelayMin=0s

// Serialization Settings
, alignment=4
, serializationByteOrder="Motorola"
, stringLengthFieldSize=4
, bom=true
, stringTermination="null"
, arrayLengthFieldSize=4
, unionLengthFieldSize=0
, unionSelectorFieldSize=4
, structLengthFieldSize=0
]
binding SOMEIP;
```



Supported Attributes (global)

Name	Type	Default	Semantics
initialFindDelayMin	Time	1s	Minimum time to randomly delay message transmission on client side.
initialFindDelayMax	Time	1s	Maximum time to randomly delay message transmission on client side.
findRepetitionsBaseDelay	Time	1s	Time for delay of repetitions on client side.
findRepetitionsMax	UInt	None	Maximum number of repetitions on client side.
findTTL	Time	5s	Time to live for service discovery messages on client side.
initialOfferDelayMin	Time	1s	Minimum time to randomly delay message transmission on server side.
initialOfferDelayMax	Time	1s	Maximum time to randomly delay message transmission on server side.
offerRepetitionsBaseDelay	Time	0s	Time for delay of repetitions on server side.
offerRepetitionsMax	UInt	None	Maximum number of repetitions on server side.
offerTTL	Time	5s	Time to live for service discovery messages on server side.
responseDelayMin	Time	0s	Delay for responses to requests.
responseDelayMax	Time	0s	Delay for responses to requests.
offerCycleTime	Time	1s	Interval between cyclic offers.
simulationChannel	UInt	1	An identifier for the simulation channel to use.

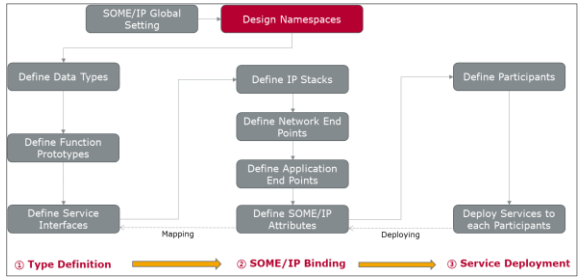
- Service Discovery -

Supported Attributes (global)

Name	Type	Default	Semantics
arrayLengthFieldSize	UInt	4	Specifies the number of bytes used to encode the size of an array datatype. Valid values: 4, 2, 1, 0
structLengthFieldSize	UInt	4	Specifies the number of bytes used to encode the size of a struct datatype. Valid values: 4, 2, 1, 0
unionLengthFieldSize	UInt	4	Specifies the number of bytes used to encode the size of a union datatype. Valid values: 4, 2, 1, 0
unionSelectorFieldSize	UInt	4	Specifies the number of bytes used to encode the size of a union selector. Valid values: 4, 2, 1
stringLengthFieldSize	UInt	4	Specifies the number of bytes used to encode the size of a string datatype. Valid values: 4, 2, 1, 0
alignment	UInt	1	Specifies the byte alignment for data behind variable length data elements. Valid values: 4,2,1
serializationByteOrder	String	"Intel"	Specifies whether values and length fields are serialized in Motorola or Intel byte order
bom	Bool	false	Specifies whether to use a byte order mark for UTF-8 and UTF-16 string datatypes.
stringTermination	Enum "null"		Specifies how UTF-8 strings are terminated, can be either "null", "ff" or "none".

- Serialization Configuration -

vCDL: Vector Communication Description Language



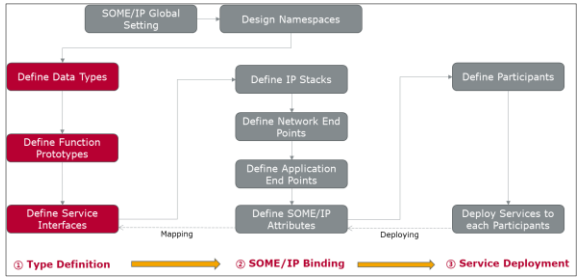
- Namespaces
  - Is the range which objects can be identified
  - Used to group in typical programming areas for example:
    - Class and class member, Functions, Variables, so on …
  - Used to group in vCDL such as:
    - Data type definitions
    - Function prototypes
    - Interfaces
    - Linear coding rules
    - Binding-specific definitions

```
/* Type Definitions */
namespace DataTypes
{
  //Type Definitions - Function Types
  typedef Add_prototype = void(in int32 opera
  typedef Subtract_prototype = void(in int32
  typedef Multiply_prototype = void(in int32
  typedef Devide_prototype = void(in int32 ope
```

```
/* Bus Systems - Ethernet Peer Setup */
namespace NetworkEndPoints
{
  //CANoe IP Stack - Consumer
  ipstack Consumer
  {
    [mac="02:00:00:00:01:FF"]
    ethernetadapter ETH
```

```
/* Binding - Autosar */
namespace CommunicationObjects
{
  [
  serviceId=1666,          //serviceId: Unique identifier for the service.
  version=0                //version: Associates a version with a service or interface.
  ]
  service Calculator : DataTypes::Calculator
  {
    //Methods
    [methodId=1]           //methodId: Unique identifier for the method.
    refine Add;
```

vCDL: Vector Communication Description Language



- Data Types
  - Primitive Data Types
    - Integer, Float, Bool, String, …
  - Complex Data Types
    - Struct, Array, Bit fields, List, …
- Function Prototypes
  - The service interface method is defined by a function prototype declaration.
  - The prototype declaration uses the `typedef` keyword, followed by the prototype name and its signature.
- Service Interfaces
  - Methods
  - Events
  - Fields

```
namespace DataTypes
{
  //Type Definitions - Function Types
  typedef Add_prototype = void(in int32 operand1, in int32 operand2, out float result);
  typedef Subtract_prototype = void(in int32 operand1, in int32 operand2, out float result);
  typedef Multiply_prototype = void(in int32 operand1, in int32 operand2, out float result);
  typedef Devide_prototype = void(in int32 operand1, in int32 operand2, out float result);

  //Type Definitions - Data Types
  [fixLayout=true]
  struct StateType
  {
    int32 AddCount;
    int32 SubtractCount;
    int32 MultiplyCount;
    int32 DivideCount;
  }

  //Type Definitions - Service Interfaces
  interface Calculator
  {
    //Mothods
    method Add_prototype Add;
    method Subtract_prototype Subtract;
    method Multiply_prototype Multiply;
    method Devide_prototype Divide;

    // Events
    event StateType State;

    // Fields
    field int32 CalcResult { get; set; notify; }
  }
}
```

Function Prototypes

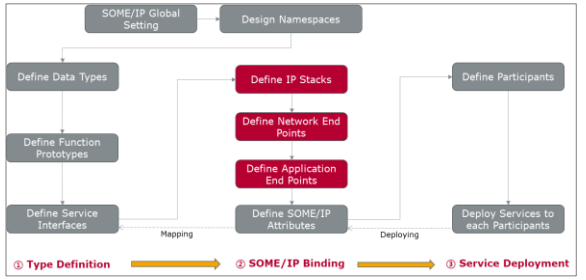
Data Types

Supported Data Types

Primitive Data Types	Complex Data Types
Strings	Enumerations
Boolean	Bit fields
Integer numbers	Arrays
Float numbers	Lists
Time	Structures
	Unions

Service Interfaces

vCDL: Vector Communication Description Language



- Endpoints
  - IP Stacks
    - Used to easily handle the IP-based communication.
    - Define MAC address as a ethernetadapter
  - Network Endpoints(NEP)
    - Refer to ethernetadapter of IP Stack
    - Define IP Address
  - Application Endpoints(AEP)
    - Refer to ipnep for binding IP Address
    - Define Port Number
    - Used as SOME/IP Binding

```
namespace NetworkEndPoints
{
  //CANoe IP Stack - Consumer
  ipstack Consumer
  {
    [mac="02:00:00:00:01:FF"]
    ethernetadapter ETH
    {
    }
  }

  //CANoe IP Stack - Provider
  ipstack Provider
  {
    [mac="02:00:00:00:00:FF"]
    ethernetadapter ETH
    {
    }
  }

  //Network Endpoints (NEPs)
  ipnep NEP_Consumer [ethernetAdapter=Consumer.ETH, address="192.168.1.2", prefix=24];
  ipnep NEP_Provider [ethernetAdapter=Provider.ETH, address="192.168.1.1", prefix=24];
  ipnep NEP_SD_MULTICAST_Consumer [ethernetAdapter=Consumer.ETH, address="239.0.0.1"];
  ipnep NEP_SD_MULTICAST_Provider [ethernetAdapter=Provider.ETH, address="239.0.0.1"];

  //Application Endpoints (AEPs)
  ipaep AEP_SD_Consumer [nep=NEP_SD_MULTICAST_Consumer, port=30490];
  ipaep AEP_SD_Provider [nep=NEP_SD_MULTICAST_Provider, port=30490];
  ipaep AEP_Consumer_UDP40000 [nep=NEP_Consumer, port=40000];
  ipaep AEP_Provider_UDP40000 [nep=NEP_Provider, port=40000];
}
```

IP Stacks

Refer

Network Endpoints

Refer

Application Endpoints

Used

SOME/IP Binding

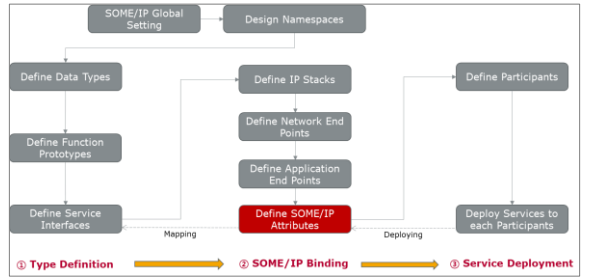
vCDL: Vector Communication Description Language

- SOME/IP Binding
  - service inherit the Service Interface
    - service Calculator : DataTypes::Calculator
  - Map the SOME/IP attributes for each service type with refine
  - Service Mapping
    - Service ID
    - Version
  - Method Mapping
    - Method ID
  - Event Mapping
    - Event ID
  - Field Mapping
    - Getter ID
    - Setter ID
    - Event ID for Field notification

```
namespace CommunicationObjects
{
  [
  serviceId=1666,
  version=0
  ]
  service Calculator : DataTypes::Calculator
  {
    //Methods
    [methodId=1]
    refine Add;
    [methodId=2]
    refine Subtract;
    [methodId=3]
    refine Multiply;
    [methodId=4]
    refine Divide;

    //Events
    [eventId=8001]
    refine State;

    //Fields
    [
    getterId=5,
    setterId=6,
    eventId=8002
    ]
    refine CalcResult;
```



Service Mapping - 'Calculator' Service
- Service ID = 1666
- Version = 0

Method Mapping - 'Add' Method
- Method ID = 1

Event Mapping - 'State' Event
- Event ID = 8001

Field Mapping - 'CalcResult' Field
- Getter ID = 5
- Setter ID = 6
- Event ID for Field notification = 8002

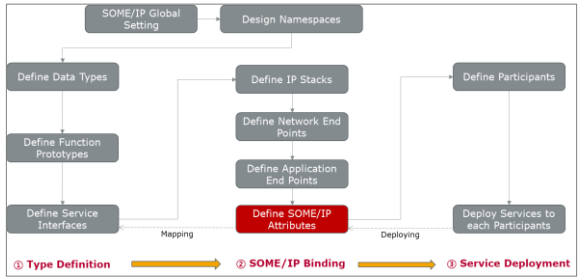
vCDL: Vector Communication Description Language

- SOME/IP Binding
  - binding *SOMEIP* service to SOME/IP
  - Define Service consumer
  - Define Service provider
  - Event Group Mapping
    - Event Group ID
  - Service Endpoint Mapping
    - Consumer Client ID
    - Provider Instance ID
    - SOME/IP Application Endpoint
    - SOME/IP SD Application Endpoint

```
//Bind the service to SOME/IP
binding SOMEIP
{
  //Event Group Definition
  [
  groupId=1,
  multicastThreshold = 0
  ]
  eventgroup EventGroup = State, CalcResult;
}

//Service Endpoint - Consumer
[
clientId=0,
udpEndpoint=NetworkEndPoints::AEP_Consumer_UDP40000,
sdMulticastEndpoint=NetworkEndPoints::AEP_SD_Consumer
]
consumer Consumer;

//Service Endpoint - Provider
[
instanceId=1,
udpEndpoint=NetworkEndPoints::AEP_Provider_UDP40000,
sdMulticastEndpoint=NetworkEndPoints::AEP_SD_Provider
]
provider Provider;
}
}
```

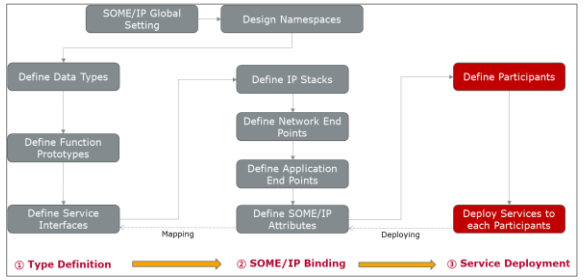


Event Group Mapping
- Event Group ID = 1
- Group = State, CalcResult

Service Endpoint Mapping - Consumer
- Client ID = 1
- SOME/IP Application Endpoint
- SOME/IP SD Application Endpoint

Service Endpoint Mapping - Provider
- Instance ID = 1
- SOME/IP Application Endpoint
- SOME/IP SD Application Endpoint

vCDL: Vector Communication Description Language



- Participants
  - Used to group communication endpoints which typically belong to one ECU or SWC.
  - participants have service endpoints.

```
namespace Participants
{
  participant Consumer
  {
    CommunicationObjects::Calculator.Consumer       //Service Endpoint - Consumer
  }

  participant Provider
  {
    CommunicationObjects::Calculator.Provider       //Service Endpoint - Provider
  }
}
```

Application Layer Objects

Participants
- Consumer
- Provider

Name	State	Role	Binding
Calculator[Provider]	Simulated (CANoe)	Provider	SOMEIP

Application Layer Objects

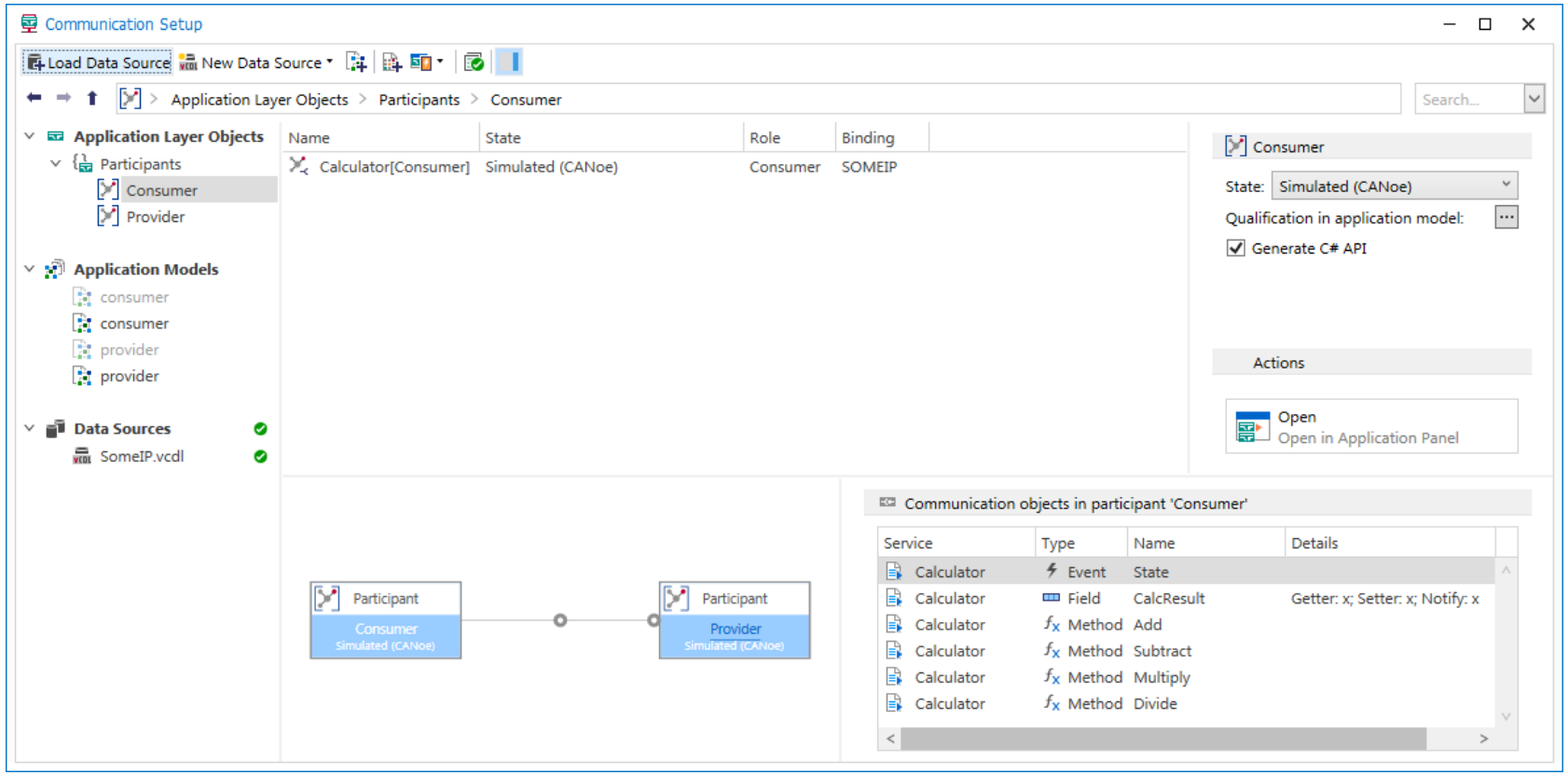
Participants
- Consumer
- Provider

Name	State	Role	Binding
Calculator[Consumer]	Simulated (CANoe)	Consumer	SOMEIP

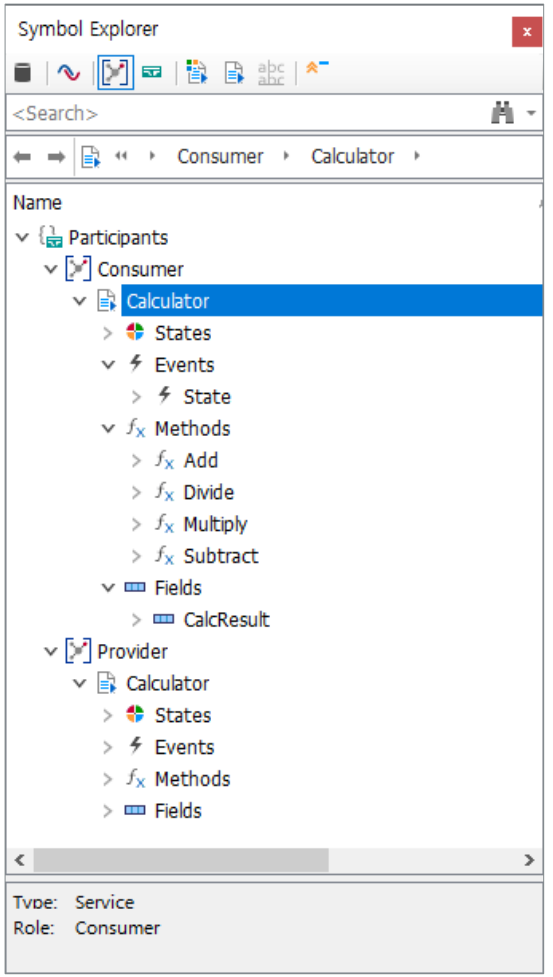
- CANoe Communication Setup -

vCDL: Vector Communication Description Language

- vCDL configured structure can be checked in …



- CANoe Communication Setup -



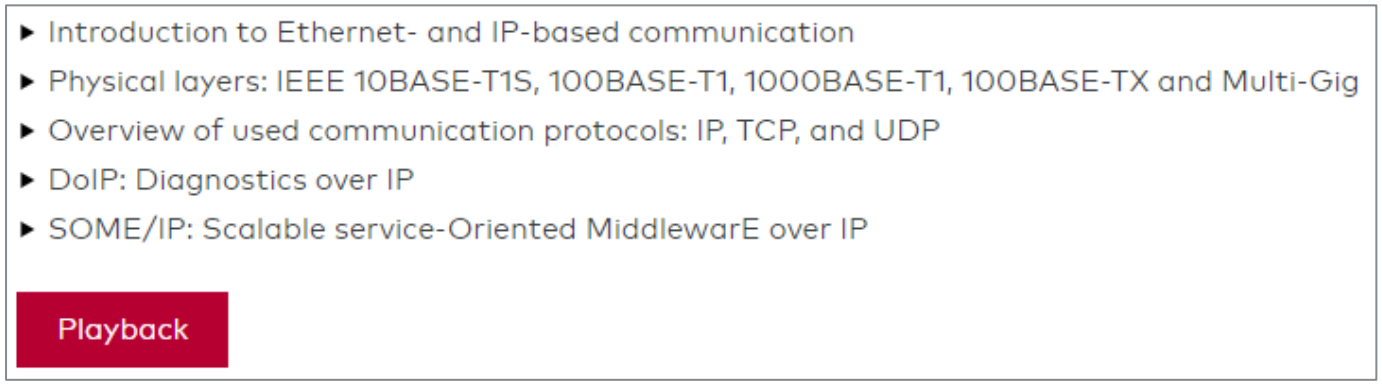
- CANoe Symbol Explorer -

Agenda

- Overview
- Data Source (vCDL)
- **Technical References**

Video Recording

- Introduction to Automotive Ethernet
  - https://www.vector.com/kr/ko/events/global-de-en/webinar-recordings/2023/introduction-to-automotive-ethernet/

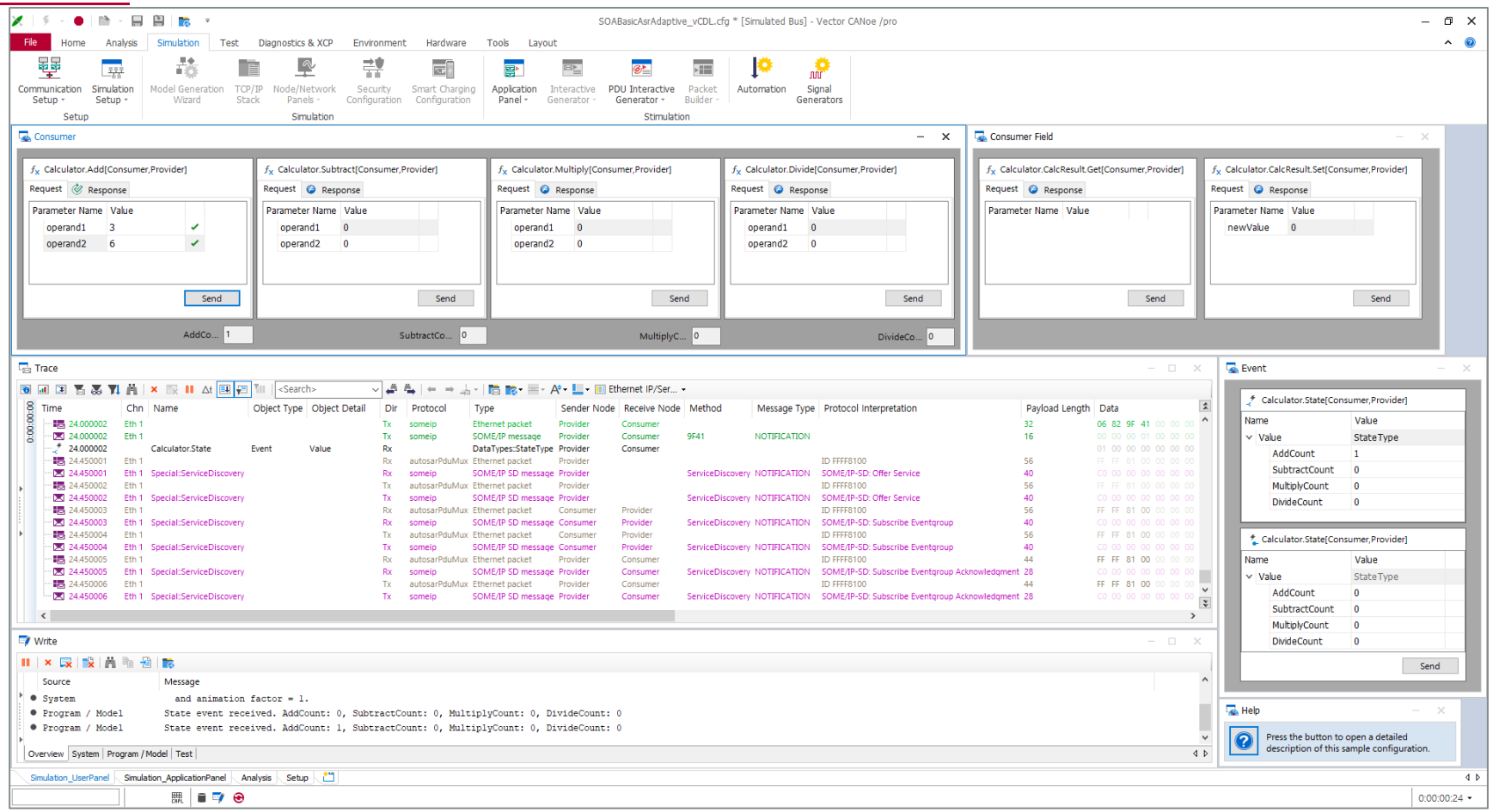


- CANoe for Service-Oriented Architectures(SOA) - Part1. SOA Fundamental
  - https://youtu.be/M_SXOgci1p4



Calculator Example

- CANoe configuration, vCDL, Analysis Features, Simulation Features, CAPL Codes
  - Download Sample Configuration : https://portal.vector.com/shared/068f1303-77de-4eab-8bd0-ca37938c6885

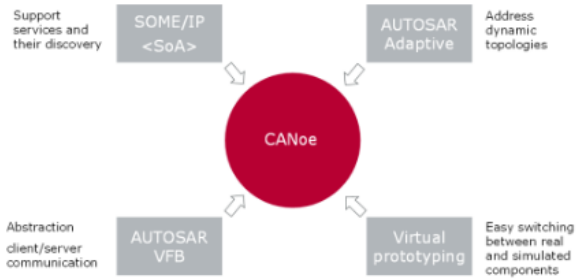


CANoe Help Manual

CANoe Communication Concept Overview

CANoe » Communication Concept

Communication Concept



CANoe as simulation and test environment provides an uniform and cross-network communication concept. This communication concept enables application-related modeling of classic and new communication patterns. The communication concept provides consistent decoupling of logical communication on the application level from protocols of the network level.
This means that CANoe provides a network-independent interface for application models and test scripts. Separately from this, the actual access or transmission is configured via a specific network.

- **Comparison**: How was it before – how is it now?

Concept

- Basics of the Communication Concept
- Service-Oriented Communication
- Binding

Application Layer Objects

- Communication Objects
- Distributed Objects
- Display and Stimulate Application Layer Objects

Configuration

- Communication Setup
  - Load Data Source
  - AUTOSAR Preprocessor
  - CANoe AUTOSAR Converter

Binding

Communication Objects

- Abstract Binding
- SOME/IP Binding

Distributed Objects

- Abstract Binding
- MQTT Binding
- HTTP Binding
- DDS Binding
- IoT Enabler
- Mapping
- CAPL
- C#
- FDX

Serialization

- JSON
- Google Protocol Buffers

Programming

- Programming C#, Python and CAPL
  Description how you can use the objects and the APIs of the communication concept in C#, Python or in CAPL to create, for example application models or to test functionality.
- Vector Communication Description Language (vCDL)
  vCDL allows the CANoe communication concept to be extended by a tool for the simple and text-based configuration of communication objects.

CANoe Help Manual

Technical Reference: vCDL

CANoe » vCDL » Language

Technical Reference: vCDL

You can use vCDL to define objects and their parameters for the CANoe communication concept.

vCDL

- Language Conventions
- Attributes
- Character Set
- Comments
- Constant Declarations
- Literals
- E2E Protection
- Expressions
- Namespaces
- Preprocessor
- References
- Secure Onboard Communication
- Type Alias
- Versioning

Data Source

Data sources for the CANoe configuration.

- Data Source – Import (vCDL)
- Data Source (Communication Setup)

Data Model

Define and edit communication/distributed objects.

- **Communication Objects (vCDL)**
  - Encodings and Semantic Attributes
  - Events
  - Fields
  - Function Prototypes
  - Methods
  - PDUs
  - PDU Timings
  - PDU Types
  - Services
  - Interfaces

Data Types

- **Primitive Data Types**
  - Boolean
  - Bytes Data Type
  - Float
  - Integer
  - String Data Type
- **Complex Data Types**
  - Array Data Types
  - Bit Fields
  - Enumerations

For more information about Vector
and our products please visit

www.vector.com



Author:
Lee, Jaecheol
Vector Korea

© 2023. Vector Korea IT Inc. All rights reserved. Any distribution or copying is subject to prior written approval by Vector. V1.0 | 2023-07-18